DESIGN AND ANALYSIS OF ALGORITHMS

ASSIGNMENT 1: Fibonacci

Question: What is the Fibonacci series?
The Fibonacci series is a sequence of numbers in which each number is the sum of the two preceding numbers. It typically starts with 0 and 1.

Question: Write the recurrence relation of Fibonacci.
The recurrence relation of the Fibonacci sequence is:
$F(n) = F(n-1) + F(n-2)$, for $n \geq 2$

Question: What are base conditions?
Base conditions are the initial values required to start a recursive definition. For Fibonacci, the base conditions are $F(0) = 0$ and $F(1) = 1$.

Question: Write the first 5 or 10 Fibonacci numbers.
First 5 numbers: 0, 1, 1, 2, 3
First 10 numbers: 0, 1, 1, 2, 3, 5, 8, 13, 21, 34

Question: What is the time complexity of recursive Fibonacci?
The time complexity of the recursive Fibonacci approach is $O(2^n)$, as it involves repeated computation of the same subproblems.

Question: Why is recursion inefficient in Fibonacci computation?
Recursion is inefficient because it recalculates the same subproblems multiple times, leading to exponential growth in function calls.

Question: What is an overlapping subproblem?
An overlapping subproblem is a problem where the same smaller subproblems are solved multiple times during execution.

Question: How does memoization improve performance?
Memoization stores the results of previously computed subproblems and reuses them when needed, reducing redundant calculations and improving time complexity to $O(n)$.

Question: What is the difference between Dynamic Programming and recursion?
Recursion solves problems by calling itself without storing intermediate results, while Dynamic Programming stores results of subproblems (using memoization or tabulation) to avoid recomputation and improve efficiency.

Question: Can Fibonacci be solved in O(log n)? How?
Yes, Fibonacci can be solved in $O(\log n)$ time using matrix exponentiation. The Fibonacci sequence can be represented using matrix multiplication, and fast exponentiation reduces the time complexity.

Sumit G.

ASSIGNMENT 1(b): Heap Sort

Question: What is Heap Sort?
Heap Sort is a comparison-based sorting algorithm that uses a binary heap data structure. It first builds a heap from the input data and then repeatedly extracts the root element to produce a sorted sequence.

Question: What is a Max Heap and a Min Heap?
A Max Heap is a binary heap in which the value of each parent node is greater than or equal to its children. A Min Heap is a binary heap in which the value of each parent node is less than or equal to its children.

Question: What is heapify?
Heapify is the process of converting a binary tree or array into a valid heap by ensuring that the heap property is maintained for all nodes.

Question: What is a complete binary tree?
A complete binary tree is a binary tree in which all levels are completely filled except possibly the last level, and all nodes are as far left as possible.

Question: Why is Heap Sort in-place?
Heap Sort is in-place because it does not require any additional memory apart from a constant amount of auxiliary space; sorting is performed within the original array.

Question: Why is Heap Sort not stable?
Heap Sort is not stable because it may change the relative order of equal elements during the heapify and swapping operations.

Question: How is a heap represented in an array?
A heap is represented using an array where for any element at index i, the left child is at index $2i + 1$ and the right child is at index $2i + 2$.

Question: Why does Heap Sort always have O(n log n) time complexity?
Heap Sort always runs in $O(n \log n)$ time because building the heap takes $O(n)$ time and each of the n extraction operations requires $O(\log n)$ time.

Question: What is the difference between Heap Sort and Selection Sort?
Heap Sort uses a heap data structure and has a time complexity of $O(n \log n)$, while Selection Sort repeatedly selects the minimum element and has a time complexity of $O(n^2)$. Heap Sort is more efficient for large datasets.

Sumit G.

ASSIGNMENT 2(a): Binary Search

Question: What is Binary Search?
Binary Search is a searching algorithm used to find an element in a sorted array by repeatedly dividing the search space into two halves and comparing the middle element with the target value.

Question: What is the prerequisite for Binary Search?
The array must be sorted in either ascending or descending order before applying Binary Search.

Question: What is the formula to find the middle index?
The middle index is calculated as: mid = low + (high − low) / 2

Question: Why is Binary Search faster than Linear Search?
Binary Search reduces the search space by half at each step, resulting in a time complexity of O(log n), whereas Linear Search checks each element sequentially with a time complexity of O(n).

Question: What happens if the array is unsorted?
Binary Search fails on an unsorted array because it relies on the sorted order to eliminate half of the search space correctly.

Question: What is the difference between iterative and recursive Binary Search?
Iterative Binary Search uses loops to repeatedly divide the search space, while recursive Binary Search uses function calls. Iterative is more space-efficient as it avoids the overhead of recursion stack.

Question: Why is mid = low + (high − low) / 2 used instead of (low + high) / 2?
The formula avoids integer overflow that may occur when low and high are large values, ensuring safe computation of the middle index.

Sumit G.

ASSIGNMENT 2(b): Quick Sort

Question: What is Quick Sort?
Quick Sort is a divide and conquer sorting algorithm that selects a pivot element, partitions the array around the pivot, and recursively sorts the subarrays on both sides.

Question: What is a pivot?
A pivot is an element selected from the array around which the partitioning process is performed, placing smaller elements on one side and larger elements on the other.

Question: What is partitioning?
Partitioning is the process of rearranging elements in the array such that elements less than the pivot are placed on one side and elements greater than the pivot are placed on the other side.

Question: Why is the worst-case time complexity $O(n^2)$?
The worst case occurs when the pivot divides the array into highly unbalanced parts, such as one element and the rest of the array, leading to n recursive calls each taking O(n) time.

Question: How can the worst case be avoided in Quick Sort?
The worst case can be avoided by choosing a random pivot or using techniques like median-of-three, which increases the chances of balanced partitioning.

Question: Why is Quick Sort faster in practice?
Quick Sort is faster in practice due to better cache performance, low constant factors, and efficient in-place partitioning compared to other algorithms like Merge Sort.

Question: Is Quick Sort stable? Why?
Quick Sort is not stable because the swapping of elements during partitioning can change the relative order of equal elements.

Sumit G.

ASSIGNMENT 3: Job Sequencing

Question: What is the Job Sequencing Problem?
The Job Sequencing Problem is an optimization problem where a set of jobs, each with a deadline and profit, must be scheduled in such a way that maximum profit is earned while completing jobs within their deadlines.

Question: What are deadlines and profits in this problem?
Deadlines represent the latest time by which a job must be completed, and profits represent the gain obtained if the job is completed within its deadline.

Question: What approach is used to solve the Job Sequencing Problem?
A greedy approach is used, where jobs are selected based on maximum profit and scheduled accordingly.

Question: Why does the greedy approach work in this problem?
The greedy approach works because selecting jobs with the highest profit first ensures maximum overall profit while still respecting the deadline constraints.

Question: How are jobs selected in the Job Sequencing Problem?
Jobs are sorted in decreasing order of profit, and each job is assigned to the latest available time slot before its deadline.

Question: What happens if no slot is available for a job?
If no time slot is available before the job's deadline, the job is skipped and not included in the final schedule.

Question: Can this problem be solved using Dynamic Programming?
Yes, the problem can be solved using Dynamic Programming, but the greedy approach is more efficient and simpler for this problem.

Sumit G.

ASSIGNMENT 4: Fractional Knapsack

Question: What is the Knapsack Problem?
The Knapsack Problem is an optimization problem where a set of items, each with a weight and value, must be selected to maximize total value without exceeding the capacity of the knapsack.

Question: What is the difference between Fractional and 0/1 Knapsack?
In Fractional Knapsack, items can be divided and taken in fractions, whereas in 0/1 Knapsack, each item can either be fully included or excluded, with no partial selection allowed.

Question: Why does the greedy approach work in Fractional Knapsack?
The greedy approach works because selecting items based on the highest value-to-weight ratio always leads to an optimal solution when fractions of items are allowed.

Question: What is the value-to-weight ratio?
The value-to-weight ratio is the value of an item divided by its weight, indicating the value gained per unit weight.

Question: Why is sorting required in Fractional Knapsack?
Sorting is required to arrange items in decreasing order of value-to-weight ratio so that the most valuable items per unit weight are selected first.

Question: Why does the greedy approach fail in 0/1 Knapsack?
The greedy approach fails because selecting items based only on value-to-weight ratio may not yield the optimal combination, as items cannot be divided and combinations must be considered, which requires dynamic programming.

Sumit G.

ASSIGNMENT 5: Bellman-Ford

Question: What is the Bellman-Ford algorithm?
The Bellman-Ford algorithm is a single-source shortest path algorithm that computes the shortest distances from a source vertex to all other vertices in a weighted graph.

Question: On what type of graph does it work?
It works on directed or undirected graphs with weighted edges, including graphs with negative edge weights.

Question: What is edge relaxation?
Edge relaxation is the process of updating the shortest distance to a vertex if a shorter path is found through another vertex using a given edge.

Question: Why is it run V−1 times?
It is run V−1 times because the shortest path in a graph with V vertices can have at most V−1 edges.

Question: How is a negative cycle detected?
After performing V−1 iterations, if any edge can still be relaxed, then the graph contains a negative weight cycle.

Question: What is the difference between Bellman-Ford and Dijkstra's algorithm?
Bellman-Ford can handle negative edge weights and detect negative cycles, whereas Dijkstra's algorithm cannot handle negative weights but is more efficient with a time complexity of O(E log V).

Question: Can Bellman-Ford handle negative cycles?
Bellman-Ford can detect the presence of negative cycles but cannot compute valid shortest paths if such cycles exist.

Sumit G.

ASSIGNMENT 6: TSP

Question: What is the Traveling Salesman Problem (TSP)?
The Traveling Salesman Problem is a combinatorial optimization problem where a salesman must visit each city exactly once and return to the starting city with the minimum possible total cost.

Question: What is the objective of TSP?
The objective is to find the shortest possible route that visits all cities exactly once and returns to the origin city.

Question: Why is brute force inefficient for TSP?
Brute force is inefficient because it generates all possible permutations of cities, resulting in factorial time complexity $O(n!)$, which becomes impractical for large n.

Question: What is the Dynamic Programming approach for TSP?
The Dynamic Programming approach uses the principle of optimality and stores solutions of subproblems using states defined by current city and visited cities, reducing time complexity to $O(n^2 \cdot 2^n)$.

Question: What is bitmasking in TSP?
Bitmasking is a technique used to represent subsets of visited cities using binary numbers, where each bit indicates whether a city has been visited or not.

Question: Why is TSP NP-hard?
TSP is NP-hard because there is no known polynomial-time algorithm to solve it, and the number of possible solutions grows exponentially with the number of cities.

Question: What are the applications of TSP?
Applications include route optimization, logistics and delivery planning, circuit design, DNA sequencing, and scheduling problems.

Sumit G.

ASSIGNMENT 7: Knapsack (Branch & Bound)

Question: What is Branch and Bound?
Branch and Bound is an optimization technique used to solve combinatorial problems by systematically exploring solution spaces and eliminating suboptimal solutions using bounds.

Question: What is bounding?
Bounding is the process of calculating an upper or lower limit (bound) on the best possible solution from a node to decide whether it should be explored further.

Question: What is pruning?
Pruning is the process of discarding nodes or branches that cannot produce a better solution than the current best, based on their bound values.

Question: What is the difference between Greedy and Branch and Bound?
Greedy algorithms make locally optimal choices without considering future consequences, while Branch and Bound explores multiple solutions systematically and guarantees a global optimal solution using bounds.

Question: How are nodes explored in Branch and Bound?
Nodes are explored using strategies such as FIFO (Breadth-First Search), LIFO (Depth-First Search), or Least Cost (Best-First Search), depending on the implementation.

Question: Why is Branch and Bound better than brute force?
Branch and Bound is better because it reduces the search space by pruning non-promising solutions, making it more efficient than brute force, which explores all possible solutions.

Sumit G.

ASSIGNMENT 8: N-Queens

Question: What is the N-Queens Problem?
The N-Queens Problem is a combinatorial problem of placing N queens on an N×N chessboard such that no two queens attack each other.

Question: What are the constraints in the N-Queens Problem?
No two queens should be placed in the same row, same column, or same diagonal.

Question: What is backtracking?
Backtracking is a recursive problem-solving technique that builds solutions step by step and abandons a solution as soon as it violates constraints.

Question: How is safety checked in the N-Queens Problem?
Safety is checked by ensuring that no queen exists in the same column, left diagonal, or right diagonal for the current position.

Question: Why is backtracking efficient for N-Queens?
Backtracking is efficient because it eliminates invalid solutions early, reducing the number of configurations that need to be explored.

Question: What is the worst-case time complexity of the N-Queens Problem?
The worst-case time complexity is O(N!), as all possible placements may need to be explored.

Question: Can multiple solutions exist for the N-Queens Problem?
Yes, multiple valid arrangements of queens can exist for a given N.

Sumit G.

VERY IMPORTANT COMMON VIVA (HIGH PROBABILITY)

Question: Define Greedy Algorithm.
A Greedy Algorithm is an approach that makes the locally optimal choice at each step with the aim of finding a global optimum solution.

Question: Define Dynamic Programming.
Dynamic Programming is a method that solves problems by breaking them into overlapping subproblems and storing their results to avoid recomputation.

Question: Define Backtracking.
Backtracking is a recursive technique that builds solutions incrementally and abandons a solution when it violates constraints.

Question: Define Divide and Conquer.
Divide and Conquer is a strategy that divides a problem into smaller subproblems, solves them independently, and combines their solutions.

Question: Explain the time complexity of Merge Sort.
Merge Sort has a time complexity of O(n log n) in all cases. The array is repeatedly divided into halves (log n levels), and merging at each level takes O(n) time.

Question: Explain one program step-by-step (Binary Search).
Step 1: Initialize low = 0 and high = n−1.
Step 2: Compute mid = low + (high − low) / 2.
Step 3: Compare the middle element with the target.
Step 4: If equal, return the index.
Step 5: If target is smaller, search in the left half (high = mid − 1).
Step 6: If target is larger, search in the right half (low = mid + 1).
Step 7: Repeat until the element is found or the search space becomes empty.

Question: Compare Greedy and Dynamic Programming.
Greedy makes local optimal choices and does not reconsider decisions, while Dynamic Programming solves all subproblems and guarantees a global optimal solution. Greedy is faster but may not always be optimal, whereas DP is more reliable but uses more time and space.

Question: Compare Recursion and Iteration.
Recursion uses function calls and a call stack to solve problems, while iteration uses loops. Recursion is more intuitive for problems like trees, but iteration is more memory-efficient.

Question: Compare Quick Sort and Merge Sort.
Quick Sort is an in-place sorting algorithm with average time complexity O(n log n) but worst-case $O(n^2)$, while Merge Sort always runs in O(n log n) but requires additional space for merging. Quick Sort is faster in practice, while Merge Sort is stable.

Sumit G.

Algorithm	Time Complexity
Fibonacci (recursive)	$O(2^n)$
Heap Sort	$O(n \log n)$
Binary Search	$O(\log n)$
Quick Sort	$O(n \log n)$, worst $O(n^2)$
Job Sequencing	$O(n \log n)$
Fractional Knapsack	$O(n \log n)$
Bellman-Ford	$O(V \times E)$
Traveling Salesman (DP)	$O(n^2 \times 2^n)$
N-Queens	$O(N!)$

Sumit G.